

Atelier ECS

- **Install AWS CLI**

- <https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>

- **Configure AWS CLI**

- Créer un nouveau utilisateur
 - AIM → create new user
 - nom : AWS-CLI-ECR
 - Fournir aux utilisateurs l'accès à la console de gestion AWS - facultatif : NO
 - suivant
 - Attacher directement des politiques :
 - AmazonEC2ContainerRegistryFullAccess
 - AmazonElasticContainerRegistryPublicFullAccess
- AIM → users → Informations d'identification de sécurité → Créer une clé d'accès
- Ouvrir la ligne CMD
 - aws configure
 - AWS Access Key ID [None]: AKIAUKYKEDEQRFLPY4R6
 - AWS Secret Access Key [None]: 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
 - Default region name [None]: eu-west-2 (region of the ECR)
 - Default output format [None]: json
 - vi ~/.aws/credentials

- **Créer le repository ECR (Docker registry)**

1. ECR → Repositories → Create repository

2. Type : Private

3. Nom : springboot-shop-prod

4. Create

- **Construire & pousser l'image Docker**

- ECR → backend référentiel → Afficher les commandes Push

- se connecter à ECR avec la commande numéro 1 (exp : `aws ecr get-login-password --region eu-west-2 | docker login --username AWS --password-stdin 297984596257.dkr.ecr.eu-west-2.amazonaws.com`) → login succeeded

- Aller dans le répertoire du projet

- construire l'image avec la commande numéro 2 (exp : `docker build -t springboot-shop-prod .`)

- vérifier que l'image est créée avec `docker images`

- Taguer l'image avec la commande numéro 3 (exp : `docker tag springboot-shop-prod:latest 297984596257.dkr.ecr.eu-west-2.amazonaws.com/springboot-shop-prod:latest`)

- vérifier avec `docker images`

- Pousser l'image dans ECR avec la commande numéro 4 (`docker push 297984596257.dkr.ecr.eu-west-2.amazonaws.com/springboot-shop-prod:latest`)

- **Créer les Security Group pour mysql RDS, et le service**

- EC2 → Groupe de Sécurité

Composant	Nom	Port Entrant	Source (Inbound)	Pourquoi ?
Service ECS	shop-service-SG	8080 / 443	0.0.0.0/0	Les clients arrivent d'Internet.
Base RDS	shop-bd-SG	3306	SG du Service ECS	Seule l'app peut interroger la DB.

- **Créer la base de données MySQL (RDS)**

- Create database
- Choisir une méthode de création de bases de données Configuration complète
- Engine : MySQL
- Template : Production (ou Dev/Test pour TP)
- DB instance identifier : backend-instance
- Master username : admin
- Gestion des informations d'identification → Autogéré
- Password : TFNStsQDcafr958
- DB instance class : db.t3.micro (TP)
- Storage : 20 GB (gp2)
- VPC : même VPC que ECS
- Public access :  No
- Subnet group : Private subnets
- Security group :
 - select shop-bd-SG
- Configuration supplémentaire
 - Nom de la base de données initiale : shop_db
 - Activer la sauvegarde automatique → no
- Create

- Créer le cluster ECS
 - ECS → Clusters → Create cluster
 - Type : Networking only (Fargate only)
 - Nom : backend-cluster
 - Create
- Créer la Task Definition
 - Launch type : Fargate
 - Name : springboot-shop-backend-task-definition
 - CPU : 1 vCPU
 - Memory : 2 GB
 - Image : ECR image URL
 - Container name : shop-backend
 - Port mapping : 8080 (Port du conteneur)

- Variables d'environnement

SPRING_PROFILES_ACTIVE=prod

DB_HOST=backend-mysql.xxxxxx.rds.amazonaws.com

DB_PORT=3306

DB_NAME=appdb

DB_USER=admin

DB_PASS=*****

JWT_SECRET=*****

- **Créer le Service ECS**

- ECS → task definition → deploy → new service
- Révision de la définition de tâche : 1
- Nom du service : springboot-shop-backend-service
- Cluster existant : backend-cluster
- Options de calcul : Type de lancement
- Type de lancement : fargate
- Stratégie de planification : Réplica
- Tâches souhaitées : 1
- Réseau → VPC : sélectionner un vpc (vpc de la BD)
- Sélectionner le groupe de sécurité shop-service-SG
- créer

- **Tester**

- ECS → Cluster → backend-cluster → springboot-shop-backend—service → Taches → configuration – Ip publique
- utiliser l’ip public pour tester l’API sur le navigateur :
<http://13.40.186.144:8080/actuator/health>
- utiliser l’ip public pour tester l’API sur postman :
<http://13.40.186.144:8080/api/auth/register>

- **Créer le Service ECS avec Load Balancer accessible via un nom de domaine**
- Modifier les sécurités groupes

Composant	Nom	Port Entrant	Source (Inbound)	Pourquoi ?
Load Balancer	shop-LB-SG	80 / 443	0.0.0.0/0	Les clients arrivent d'Internet.
Service ECS	shop-service-SG	8080	SG du Load Balancer	Seul le LB transmet les requêtes à l'app.
Base RDS	shop-bd-SG	3306	SG du Service ECS	Seule l'app peut interroger la DB.

- ECS → task definition → deploy → new service
- Révision de la définition de tâche : 2
- Nom du service : springboot-shop--service-lb
- Cluster existant : backend-cluster
- Options de calcul : Type de lancement
- Type de lancement : fargate
- Stratégie de planification : Réplica
- Tâches souhaitées : 2
- Load balancer
 - Utiliser l'équilibrage de charge : ok
 - type : Application Load Balancer
 - create load balancer (EC2)

- name : backend-shop-service-LB
- container : backend-shop 8080:8080
- new listener : http 80
- target group :
- health-check : /actuator/health
- health check grace : 20
- créer
- Mettre à jour le sécurité group du load balancer
 - security group = shop-LB-SG!
- **Tester**
 - EC2 → Équilibreurs de charge → backend-shop-service-LB → Nom du DNS
 - utiliser Nom du DNS pour tester l'API sur le navigateur : <http://backend-shop-service-lb-2061258442.eu-west-2.elb.amazonaws.com/actuator/health>
 - utiliser l'ip public pour tester l'API sur postman : <http://backend-shop-service-lb-2061258442.eu-west-2.elb.amazonaws.com/api/auth/register>
- **Utilisez un nom de domaine**
 - route 53 → zone hébergées → créer enregistrement qui pointe sur le load balancer
 - Nom de l'enregistrement : mondomaine
 - Type d'enregistrement : A - Achemine le trafic vers une adresse IPv4 et certaines ressources AWS.
 - Alias : ok
 - Acheminer le trafic vers : Alias vers l'application et Classic Load Balancer
 - Europe (Londres) la région utilisée)
 - Créer

- tester en utilisant le domaine
-
- **Utilisez un custom domaine avec https**
 - AWS Certificate Manager → Certificats → demander
 - Demander un certificat public
 - suivant
 - Noms de domaine : mdi-lab.com
 - Noms de domaine : api.mdi-lab.com
 - Désactiver l'exportation
 - Validation DNS – Recommandé
 - Demander
 - **confirmer le nom de domaine en créant des enregistrements sur Route 53**
 - • AWS Certificate Manager → Certificats → certificat id → Création d'enregistrements dans Route 53
 - **Ajouter un nouveau listener dans le load balancer**
 - EC2 → Équilibreurs de charge → backend-shop-service-LB → Écouteurs et règles → Ajouter un écouteur
 - https, 443,
 - Action de pré-routage : Aucune action de pré-routage
 - Action de routage : Faites suivre au groupe cible
 - Groupe cible : Backend-shop-service-TG
 - Certificat de serveur SSL/TLS par défaut → Source du certificat : Depuis ACM

- Certificat (d'ACM) : sélectionner le certificat pour le nom de domaine
- Ajouter
- **Rediriger toutes les requêtes http en https**
 - EC2 → Équilibreurs de charge → backend-shop-service-LB → Écouteurs et règles → HTTP:80 → Gérer les règles → modifier les règles → Dernier (par défaut) → modifier la règle
 - Action de routage : Rediriger vers l'URL
 - Protocole : HTTPS
 - Port : 443
 - Enregistrer
 - Tester avec <https://api.mdi-lab.com/actuator/health>
 - Tester avec <http://api.mdi-lab.com/actuator/health> est redirigé vers <https://api.mdi-lab.com/actuator/health>
- **Créer le Service ECS avec auto scaling**
 - Amazon Elastic Container Service → Clusters → backend-cluster → Services → springboot-shop—service-lb → Mettre à jour
 - Tâches souhaitées : 1
 - Service Auto Scaling → Mise à l'échelle automatique du service : ok
 - Nombre minimal de tâches : 1
 - Nombre maximal de tâches : 3
 - Ajouter des stratégies de mise à l'échelle
 - Nom de la stratégie : springboot-shop-backend-service-autoscaling
 - Mesure de service ECS : ALBRequestCountPerTarget
 - Mettre à jour